WAT2133C

Web Application Development

Week 1 – Overview of the .NET Framework & Programming with C#

Overview of the ASP.NET Framework

- Installing Visual Studio
- ASP.NET and the .NET Framework – an overview
- Understanding ASP.NET Page Structure
- VS IDE
- Demo

Installing VS2013/15/17/19/22

- Visual Studio can target: [Compatibility]
  - ASP.NET 2.0
  - ASP.NET 3.0
  - ASP.NET 3.5
  - ASP.NET 4.0
  - ASP.NET 4.5
  - ASP.NET 4.5.1, 4.5.2, …4.8



  - To download and install Visual Studio Community
    - https://visualstudio.microsoft.com/downloads/

Select: ASP.NET and Web Development option

ASP.NET Versions

- 1.0 – the initial release (2002)
- 1.1 – included a new version of Visual Studio .NET; no major changes in ASP.NET (2003)
- 2.0 – a wide array of new ASP.NET features and functionality, along with many new features in the .NET Framework; included new versions of Visual Studio and SQL Server (2005)
- 3.0 / 3.5
- 4.0
- 4.5
- 4.5.1 / 4.5.2
- 4.6,
- 4.6.1,
- 4.7.2, 4.8
- Versions: https://goo.gl/VpOkJy

What is ASP.NET?

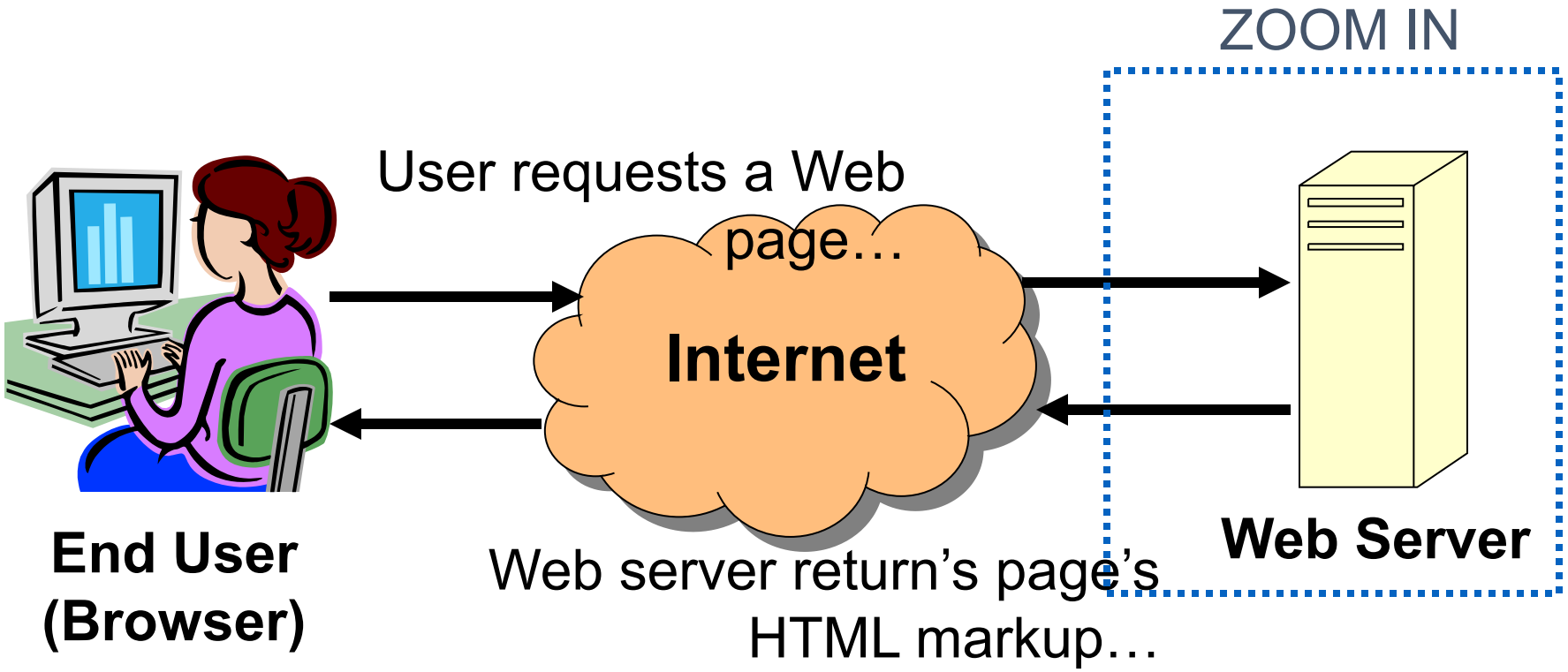
- **ASP**.**NET** is an open-source, server-side web-application framework designed for web development to produce different types of data-driven applications
- Any .NET programming language can be used (C#, VB)
- Object-oriented model
- Separation of code and UI
- Session management
- Debugging
- Embedding external libraries in VS
- and much more…

Dynamic Web Applications

- Job Portal
- E-Auction
- Blood Donation Online System
- Online Scrabble Platform
- Online Farmer’s market
- Bartering Online system
- Social Supermarket
- UTM Alumni Online System
- Stock Exchange Online System
- E-commerce System
- AI-Driven Online Tutoring System

Client/Server Architecture for Web Applications

- Let's "zoom in" on the sequence of a Web request



ASP.NET and the .NET Framework

- To build ASP.NET pages, you need to take advantage of the features of the .NET Framework.

- The .NET Framework consists of two parts:

  - the FCL and
  - the CLR.

Understanding the Framework Class Library

- The .NET Framework contains thousands of classes that you can use when building an application.
- The Framework Class Library was designed to make it easier to perform the most common programming tasks.
- Here are just a few examples of the classes:
  - Random class
  - Graphics class
  - SmtpClient class
  - File class

```
using System.Data;
```

Understanding the Framework Class Library

- The .NET Framework includes around 18,619 types; 12,909 classes; 401,759 public methods; 93,105 public properties; and 30,546 public events.
- Each class in the Framework can include properties, methods, and events. For example, here is a partial list of the members of the **SmtpClient** class:
- Properties
  - Host: The name or IP address of your email server
  - Port: number of the port to use when sending email message
- Methods
  - Send: Enables you to send an email message synchronously
  - SendAsync: send an email message asynchronously
- Events
  - SendCompleted: Raised when an asynchronous send operation completes

- The `HttpApplication` Object
- The `HttpRequest` Object
- The `HttpResponse` Object
- The Session Object
- The `Server` Object

Understanding Namespaces

- A namespace is simply a category. For example, all the classes related to working with the file system are located in the **System.IO** namespace.
- All the classes for working with a Microsoft SQL Server database are located in the **System.Data.SqlClient** namespace.
- Before you can use a class in a page, you must indicate the namespace associated with the class. There are multiple ways of doing this.

Understanding Namespaces

- First, you can fully qualify a class name with its namespace.
- For example, because the File class is contained in the System.IO namespace, you can use the following statement to check whether a file exists:

```
System.IO.File.Exists("SomeFile.txt")
```

Understanding Namespaces

- Specifying a namespace each and every time you use a class can quickly become tedious. A second option is to import a namespace.
- You can add an <%@ Import %> directive to a page to import a particular namespace:

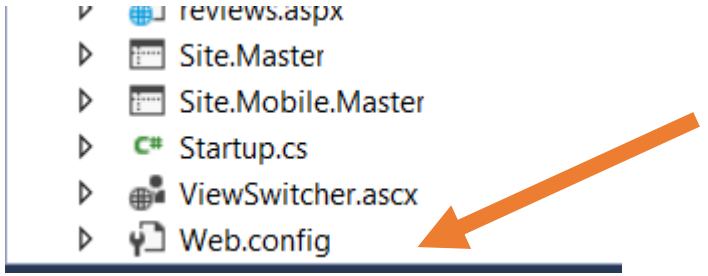
<%@ Import Namespace="System.IO" %>

or

```
using System.IO;
```

Understanding Namespaces

- If you discover that you are using a namespace in multiple pages in your application, then you can configure all the pages to recognize the namespace only once.



- A web configuration file is a special type of file that you can add to your application for configurations.

Understanding the Common Language Runtime

- The second part of the .NET Framework is the Common Language Runtime (CLR).
- The Common Language Runtime is responsible for **executing the application code**.
- When you write an application for the .NET Framework with a language such as Visual Basic .NET or C#, your source code is never compiled directly into machine code.
- Instead, the Visual Basic or C# compiler converts your code into a special language named CIL.

Understanding the Common Language Runtime

- When your application actually executes, the CIL code is "just-in-time" compiled into machine code.

- Normally, your entire application is not compiled from CIL into machine code. Instead, **only the methods** that are actually called during execution are compiled.

More about CLR

- The CLR:
  - manages execution of .NET programs
  - coordinates code execution, security and debugging processes
  - manages other aspects of the execution process such as memory management

Understanding the ASP.NET Page Structure

ASP.NET page extension: .aspx

- Directives
- Server-side Code [optional]
- Layout

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="datetime.aspx.cs"
Inherits="Tutorials_Week1_datetime" %>

<!DOCTYPE html>
<script runat="server">
   void Page_Load()
   {
      lblServerTime.Text = DateTime.Now.ToString();
   }
</script>

<html>
<head runat="server">
   <title></title>
</head>
<body>
   <form id="form1" runat="server">
      <div>
         The current date and time is:
         <asp:Label
            ID="lblServerTime"
            runat="server" />

      </div>
   </form>
</body>
</html>
```

Code-Behind Programming - .aspx.cs

- The code-behind programming model provides clean separation between the UI and the server-side code.
- In addition, the code-behind programming model adds another file to the web page called the code-behind page, which contains the server-side code: .aspx.cs

```
using System;
using System.Web;

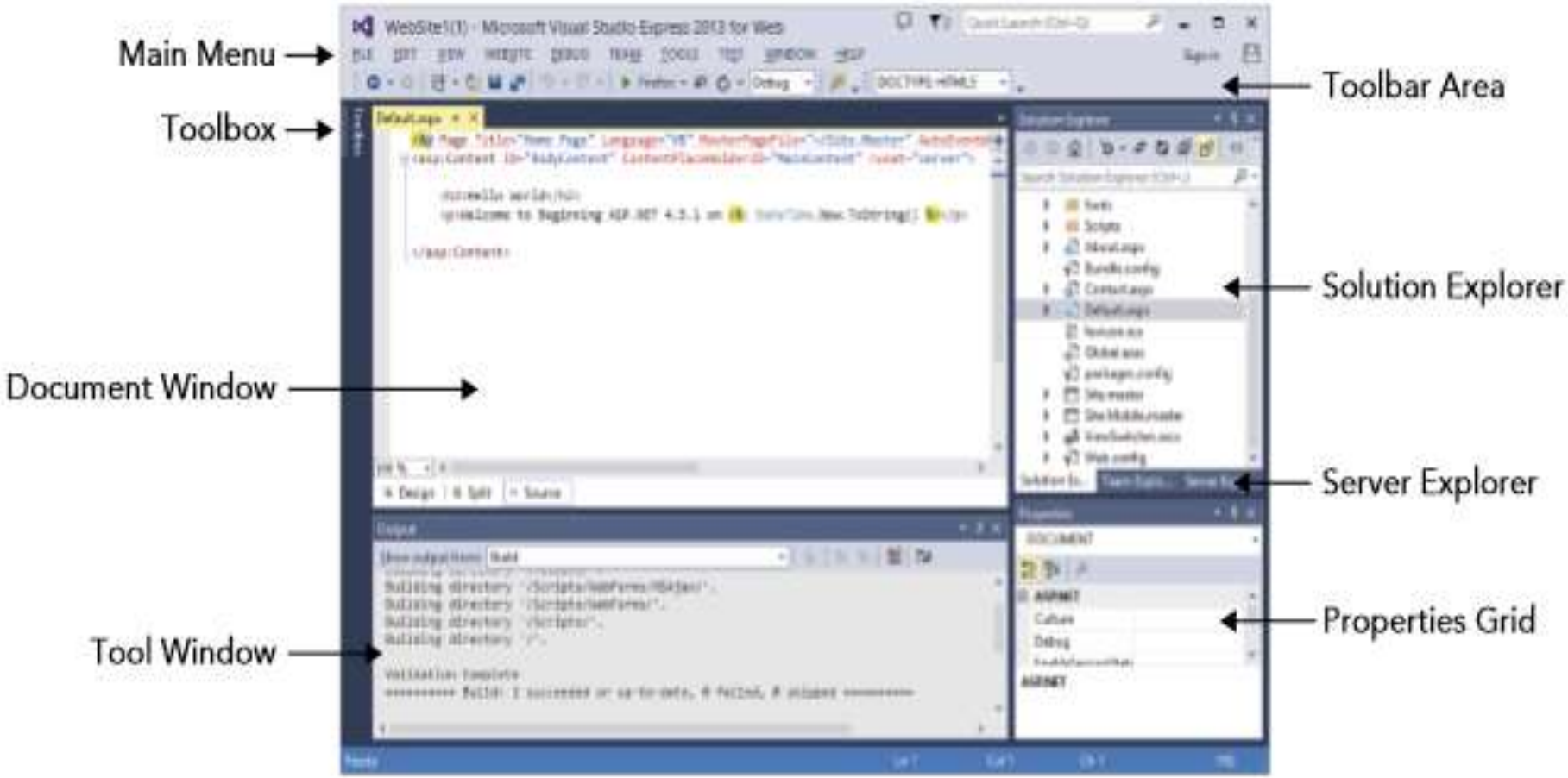
namespace MyStuff
{
   public partial class MyClass : Page
   {

      protected void Page_Load(Object sender, EventArgs e)
      {
          lblServerTime.Text = DateTime.Now.ToString();
      }
   }
}
```

Page State in .NET Web Forms

- **Dynamic Page Generation**
  - Each request creates the page anew.
- **Stateless Nature of Web Pages**
  - By default, pages and variables do not retain values between requests.
- **State Definition**
  - "State" refers to the data or values stored in variables/objects across requests.
- **Techniques to Preserve State:**
- **View State**: Client-side, maintains control values between postbacks.
- **Session State / Application State**: Server-side, stores user or app-level data.
- **Cookies**: Client-side, small text files saved on the browser to persist data.

Visual Studio IDE



Demo: welcome.aspx

- Let's create our first Web page using Visual Studio

  - ❖ Add a dummy masterpage named **dummymaster.master**
  - ❖ Add a heading <h1> within a <header>
  - ❖ Next, create a Web Page named **welcome.aspx**
  - ❖ Modify title of the page
  - ❖ Look at the page in Design and Split modes
  - ❖ To add controls to the page, you can drag and drop them from the **Toolbox** onto the Web Form in **Design** mode.
  - ❖ Add a **`Label`** control from the *Toolbox*
  - ❖ Change ID of the **`Label`** control to lblResult
  - ❖ Change some physical properties of the **`Label`** control such as its Backgroundcolor to Black

Demo: welcome.aspx

- Select the Label and press F4 to open the **Properties** window
- Change the forecolor (fuchsia) of the Label and set a width and height

In Code-Behind:

- Add a `Page_Load` event to set the `Text` property of the **`Label`** control to "**Welcome to Visual Studio IDE**"

Note:

- Page Directive <%@ … %>
- Intellisense
- The Label generates a tag

Demo: ***Running the Program***

- You can view the Web Form several ways:
  - You can select **Debug > Start Without Debugging**, which runs the application by opening it in a browser window.
  - To debug your application, you can select **Debug > Start Debugging (F5)**. You cannot debug a web application unless debugging is explicitly enabled by the **web.config** file.
  - To run a web page, you can right click either the Web Form Designer or the ASPX file name and select **View In Browser**.
  - Finally, you can run your application by opening a browser window and typing the web page's URL in the **Address** field.

Demo: Event Handling [welcome.aspx]

- Open the welcome.aspx Web page
  1. **Add a `Button` (`btnDateTime`) with foreground color fuchsia**
  2. To create this click event handler, double click the `Button` on the `Form`.
  3. Note: an empty event handler is created
  4. Set the `Text` property of the `Label` control with the current date time.

- To add an event handler, alternatively in markup (aspx) file:
  1. Add an `onclick=" "` property to the `Button` control.
  2. Choose CreateNewEvent option

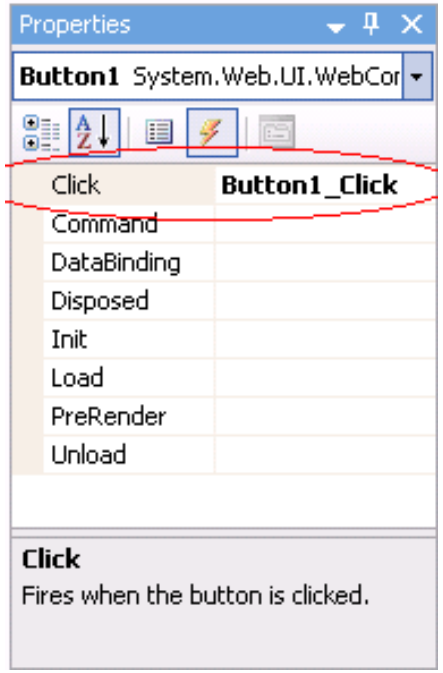
Event Handling

- By convention, the event-handler method name is set to *controlid_eventName* (e.g., `btnDateTime_Click`).
- Each event handler receives two parameters when it is called:
  - An object reference named `sender`—a reference to the object that generated the event.
  - A reference to an object of type `EventArgs`, which contains additional information about the event.

```
protected void btnDateTime_Click(object sender, EventArgs e)
  {
     lblresult.Text = DateTime.Now.ToString();
  }
```

Other Ways to Create Event Handlers

- Typically, controls can generate many different types of events.
- Clicking the **Events** icon (the lightning-bolt icon) in the **Properties** window, displays all the events for the selected control.



HW: add the following controls and double click on them to generate their event:

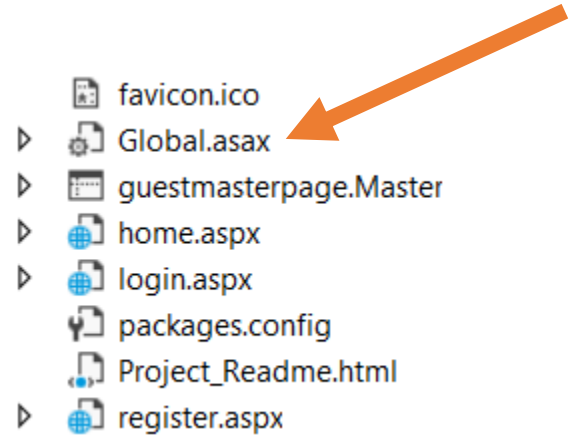
- TextBox
- RadioButton
- DropdownList
- LinkButton
- ImageButton
- Button

Role of global.asax in Life Cycle Events

- Global.asax file is created at the root directory of the application.
- This file contains the code to handle application events.

```
void Application_Start(object sender, EventArgs e)
  {

  }
```



Recap

- What are the three important parts of an ASP.NET web page?
- As compared to in-line coding, what are the advantages of code-behind programming model?
- Give three ways how to invoke namespaces.
- How CLR works?
- What is FCL?
- What do you understand by the term “Stateless”?
- Explain the importance of ViewState in a Web Page and how it works?
- Name the file used to add configuration in a website.